Machine Learning For Scientific Discovery

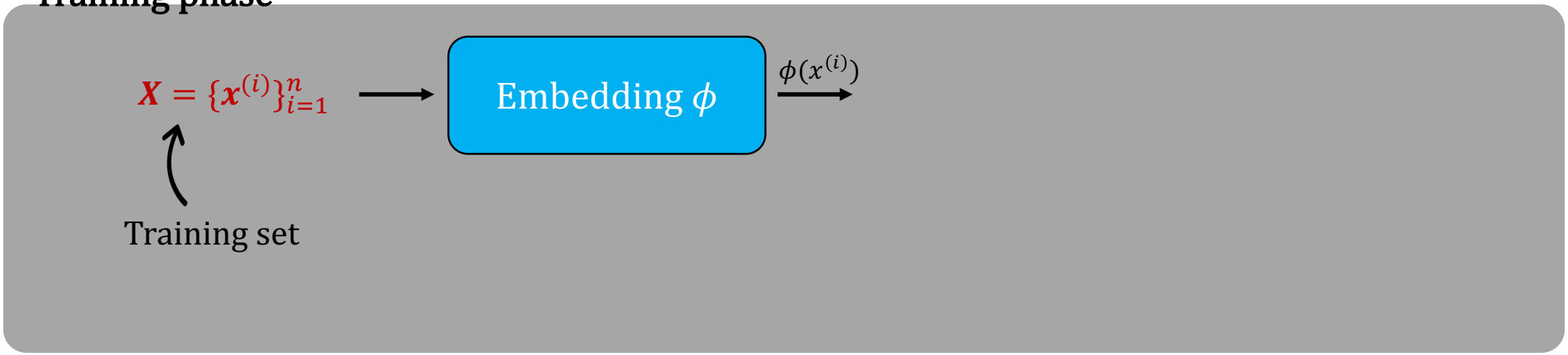
Tony Bonnaire

Image generated with Midjourney « A representation of deep learning merging with chemistry »

LPENS ENS | PSL

ML building blocks

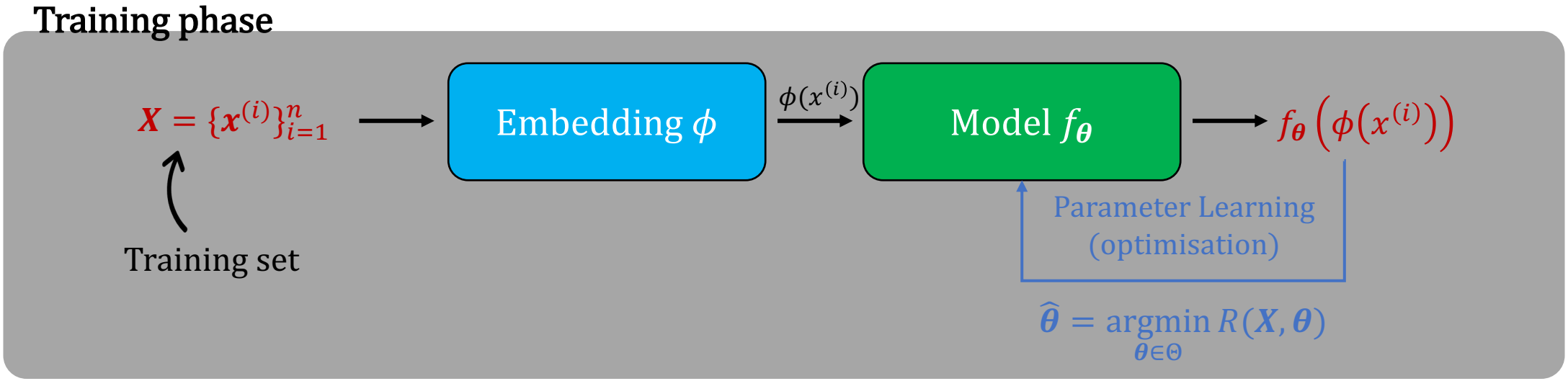
Training phase



1. Data X are **unstructured**, sometimes **noisy** and **unprocessed** like pixels of an image or sequence of characters or words.
2. The embedding $\phi(x^{(i)})$ is a **structured**, **numerical** vector representation of the data whose elements are **meaningful features**. It depends on the data and the purpose. It can be **handcrafted or learnt**.

Finding a good embedding is a central part of ML: it eases the problem by preserving the essential structure of the data that matters for the task.

ML building blocks



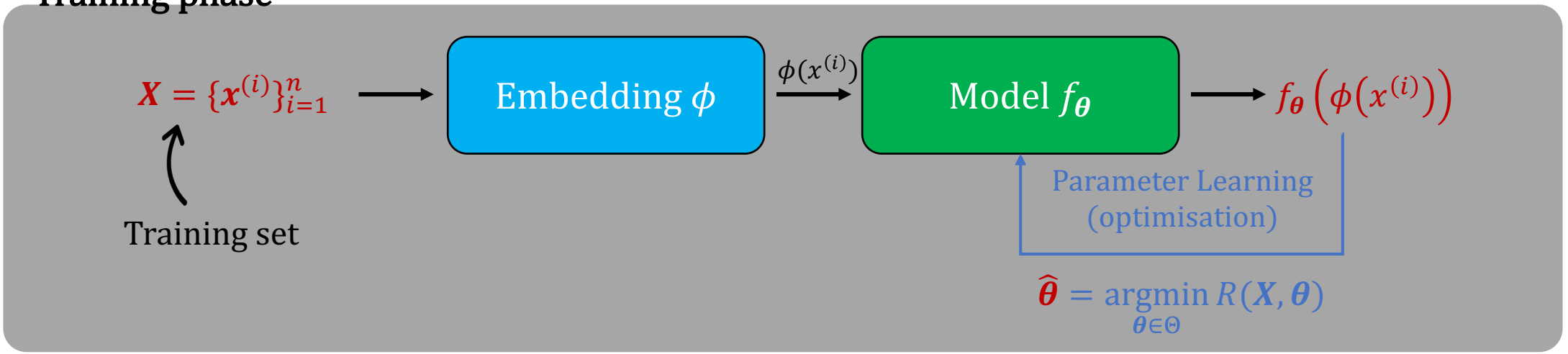
Some notations and terminologies:

- $x^{(i)} \in \mathbb{R}^d$ is *one* ***training data*** (there are n of them),
- $\phi\left(x^{(i)}\right) \in \mathbb{R}^{d'}$ is an embedding of $x^{(i)}$ sometimes called ***feature vector,***
- $\theta \in \Theta \subset \mathbb{R}^p$ are the ***parameters*** of the model,
- $R(X, \theta)$ is the ***risk*** and measures the error of the model with parameters θ on data X.

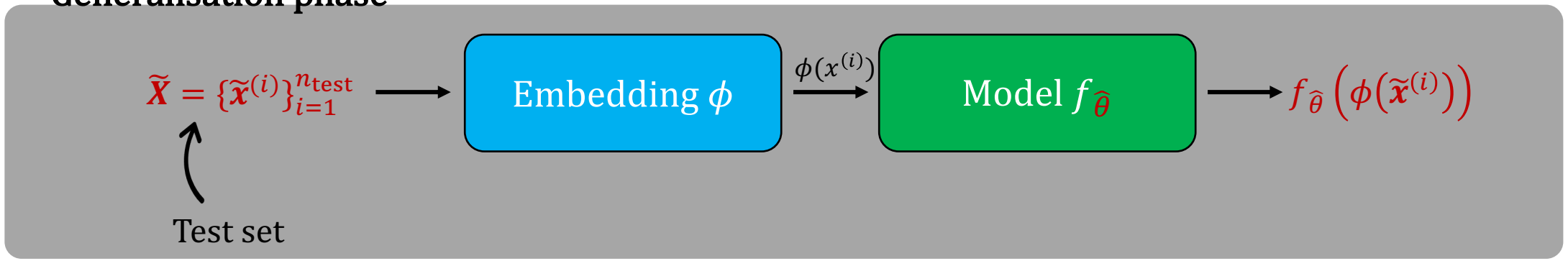
At the end of the training procedure, we have a model $f_{\hat{\theta}}$ committing an error of $R_{\text{train}} = R(X, \hat{\theta})$ on the training set.

ML building blocks

Training phase



Generalisation phase



Using the test set, we can evaluate the test error $R_{\text{test}} = R(\tilde{X}, \hat{\theta})$ and compare it to R_{train} to detect **generalisation issues** (**overfitting** or **underfitting**).

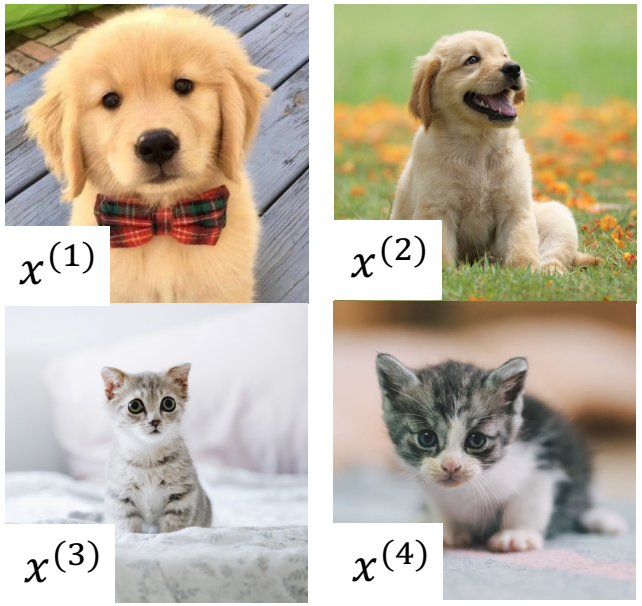
Supervised learning

$\boldsymbol{x}^{(i)}$ is the ith **data vector** of the training base, and $y^{(i)}$ is called the **target (or predicted) variable**

- Training data are actually X and Y coming as pairs

$$\boldsymbol{X} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{n}, \qquad (\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}) \in \mathbb{X}\times\mathbb{Y}$$

- If $\mathbb{Y}$ is continuous, then the task is called **regression**, and if $\mathbb{Y}$ is discrete, then it is a **classification** problem.

$X =$  $x^{(1)}$ $x^{(2)}$ $x^{(3)}$ $x^{(4)}$

Example: Determine if an image encodes a cat or a dog (called a **classification** task)

$Y = \{1, 1, 0, 0\}$ $\quad f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) = \hat{\boldsymbol{y}}^{(i)}$ $\longrightarrow$ $f_{\hat{\theta}}$

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, R(\boldsymbol{X}, \boldsymbol{\theta})$$

Training data | Model | Optimisation

Supervised learning

- Training data are actually X and Y coming as pairs

$$\boldsymbol{X} = \left\{\left(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}\right)\right\}_{i=1}^{n}, \qquad \left(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}\right) \in \mathbb{X}\times\mathbb{Y}$$

- If $\mathbb{Y}$ is continuous, then the task is called **regression**, and if $\mathbb{Y}$ is discrete, then it is a **classification** problem.

- Ideally, we would like to minimise the **expected risk**, i.e. the **expected value of a loss function** $\ell(y, \hat{y})$

$$R(\boldsymbol{X}, \boldsymbol{\theta}) = \mathbb{E}_{X,y}[\ell(y, \hat{y})]$$

Loss function: measures how bad your model is on a single example

However, we do not know $p(X, y)$ so in practice we rely on the **empirical risk** instead

$$\hat{R}(\boldsymbol{X}, \boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(y^{(i)}, \hat{y}^{(i)}\right).$$

Families of ML problems

Supervised learning

- Training data are actually X and Y coming as pairs

$$\boldsymbol{X} = \left\{\left(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}\right)\right\}_{i=1}^{n}, \qquad \left(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}\right) \in \mathbb{X}\times\mathbb{Y}$$

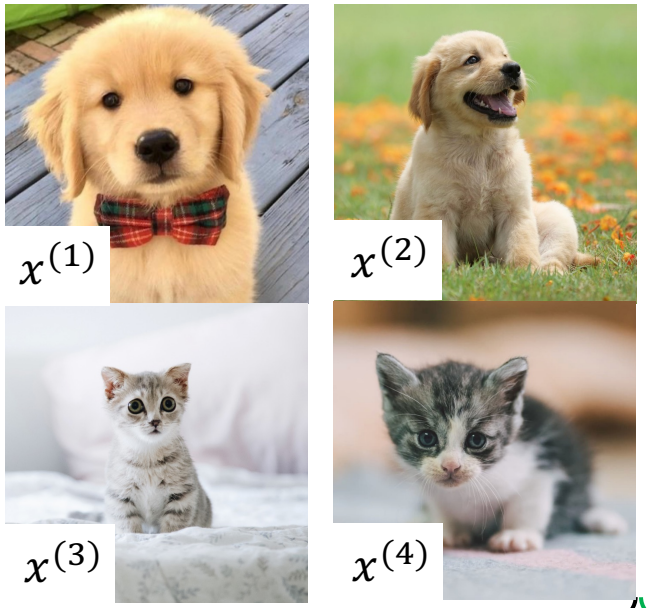
- If $\mathbb{Y}$ is continuous, then the task is called regression, and if $\mathbb{Y}$ is discrete, then it is a classification problem.

Examples of tasks: Classification | Regression | Timeseries prediction | Segmentation

Examples of models: Artificial Neural network | Random forest | Linear regression | Logistic regression | Naïve Bayes | Nearest neighbours

Unsupervised learning

- Training data are the set of $\boldsymbol{x}^{(i)}$'s only; no known results to predict
- In unsupervised learning, one seeks **patterns or structures** in X without prior labels
- Usually boils down to model the probability distribution of the dataset

$X =$ 

Training data

$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = p_{\theta}(\boldsymbol{x})$ ⟶ $f_{\hat{\theta}}$

$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} R(\boldsymbol{X}, \boldsymbol{\theta})$

such that $p_{\theta}(\boldsymbol{x}) \approx p(\boldsymbol{x})$

Model

Optimisation

Example: Generate new images of cats and dogs (called a **generation** task)

Families of ML problems

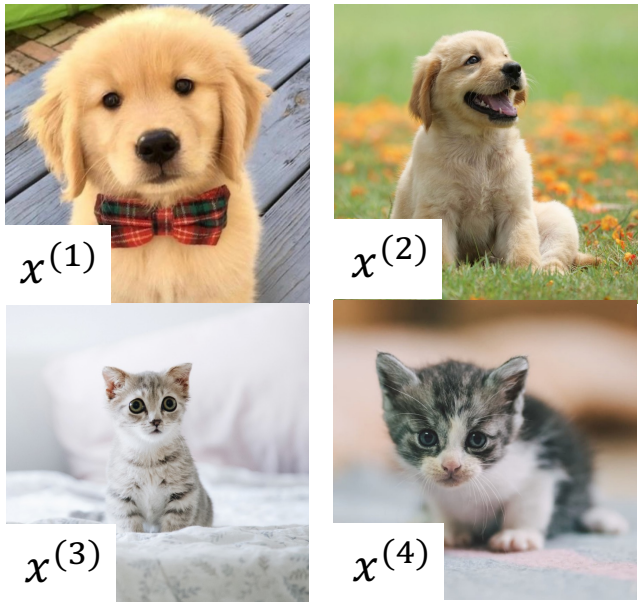
Unsupervised learning

- Training data are the set of $\boldsymbol{x}^{(i)}$'s only; no known results to predict
- In unsupervised learning, one seeks **patterns or structures** in X without prior labels
- Usually boils down to model the probability distribution of the dataset

Examples of tasks: Clustering | Data augmentation | Dimensionality reduction | Sampling

Examples of models: Autoencoder | Boltzmann Machine | Diffusion models | Gaussian mixture model | Generative Adversarial network

Why do we need ML?



$$d \approx 10^6$$

- To sample a $[0,1]^d$ space with a shortest distance to a test point at most ϵ, we need $n_{\text{train}} \geq \epsilon^{-d} = e^{-d \log \epsilon}$
- **$d \approx 80$ requires more samples than the number of atoms in the universe**

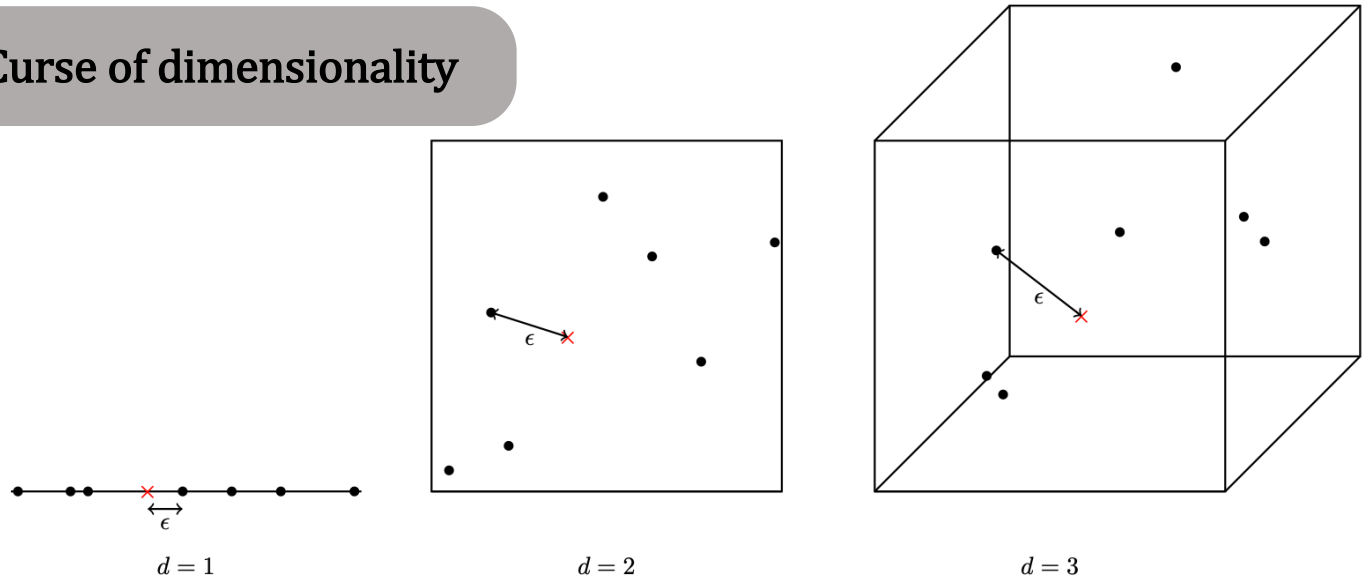
1-nearest neighbour

- A simple classification rule is for instance associating to a data the label of its closest neighbour in the d-dimensional space.

$$\hat{y} = y^{(m)} \text{ with } m = \text{argmin}_i \left\| x - x^{(i)} \right\|_2^2$$

- **In this case $R_{\text{train}} = 0$ but R_{test} is very large! Why?**

Curse of dimensionality



Why do we need ML?

Traditional methods typically break down in high-dimensional spaces (**curse of dimensionality**) and it is impossible to design handcrafted decision rules for complex tasks.

IF THERE IS ONLY ONE THING TO REMEMBER FROM THIS CLASS

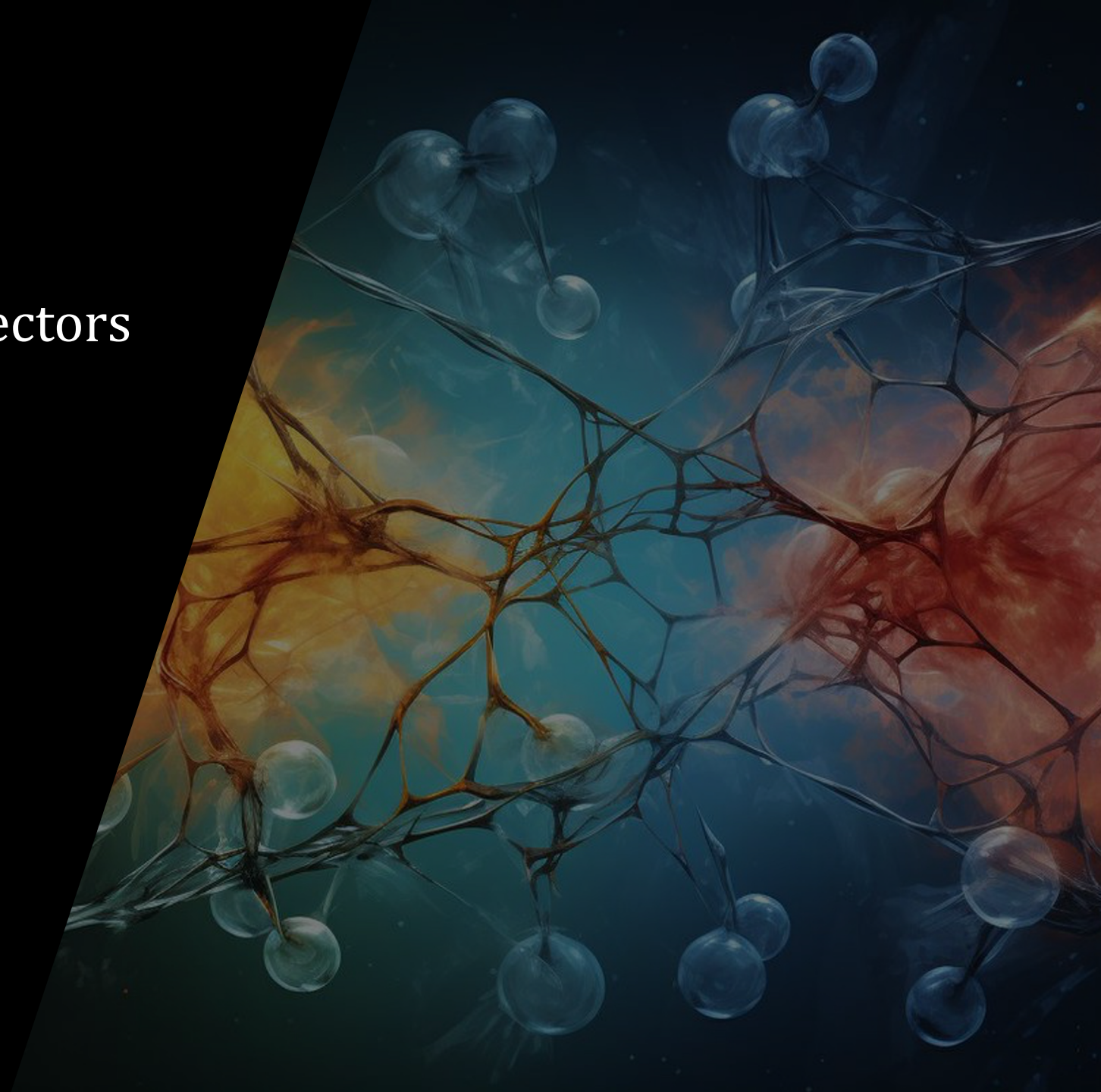
The **curse of dimensionality** is the **central problem of machine learning.** To fight it, ML relies on **prior information** about the problem:

- **Reduce the dimensionality**: select a subset of meaningful features (or their interactions) through appropriate embeddings.
- Exploit **structures** in the data (invariances, sparsity, long/short range correlations, etc.) to define the model,
- **Penalise** complex models leading to poor generalisation performances using regularisation.

I – Linear models on feature vectors

Contents:

- *Linear regression model*
- *L2-loss for regression and normal equations*
- *Linear classification model*
- *Softmax function, cross-entropy loss for classification*



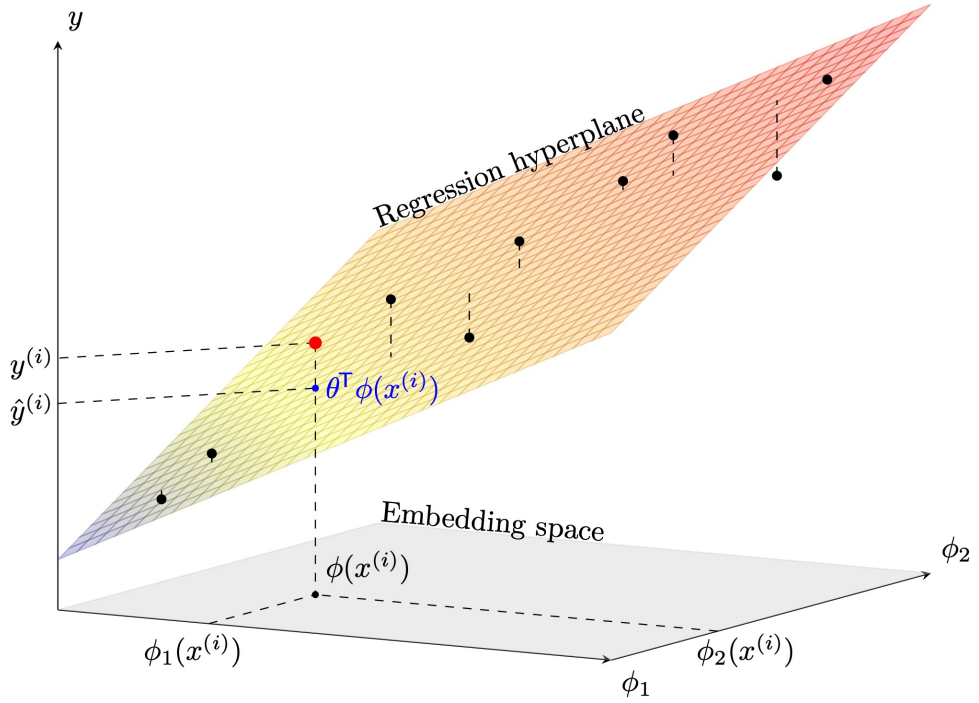
Linear regression: the model

Linear regression

- What kind of problems one can solve efficiently, even in large dimensions? → **Linear systems!**
- Let us talk first about regression: the answer is modelled as

$$f_{\boldsymbol{\theta}}\left(\phi_1^{(i)}, \phi_2^{(i)}, \cdots\right) = \boldsymbol{\theta}^{\mathrm{T}}\boldsymbol{\phi}^{(i)} = \hat{y}^{(i)}$$

- It is sometimes convenient to add an affine term (also called **bias** in the neural network literature), which can be absorbed in the feature vector making it of dimension $d' + 1$ where $\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \cdots]^{\mathrm{T}}$, $\boldsymbol{\phi}^{(i)} = \left[1, \phi_1^{(i)}, \phi_2^{(i)}, \cdots\right]^{\mathrm{T}}$.
- **Geometric interpretation**: projection of an embedding vector onto a **hyperplane** parameterised by $\boldsymbol{\theta}$.



Linear regression: example and ERM

Linear regression

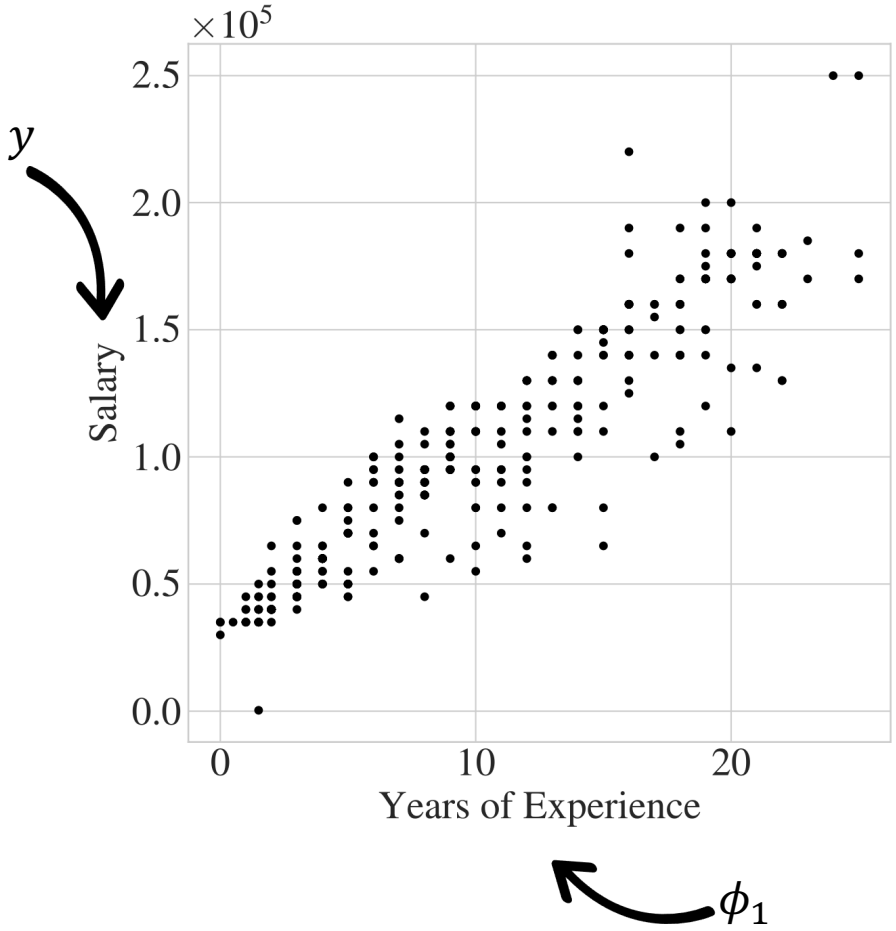
- Example: salary prediction based on the years of experience
- Data are 373 couples $\left(\phi^{(i)}, y^{(i)}\right) \Longrightarrow$ **Supervised learning**
- The target variable $y \in \mathbb{R}$ is continuous $\Longrightarrow$ **Regression**
- The linear model is

$$\hat{y}^{(i)} = \theta_0 + \theta_1 \phi_1^{(i)},$$

where $\phi_1^{(i)}$ is the nb. of years of experience of the i^{th} training example

- Now the model is fixed, how to find $\widehat{\boldsymbol{\theta}}$, the best possible parameters for our model and data?
- This is done using **empirical risk minimisation (ERM)**

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, R(\boldsymbol{X}, \boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, \frac{1}{n} \sum_{i=1}^{n} \ell\left(\hat{y}^{(i)}, y^{(i)}\right)$$



Linear regression: solution to ERM

Linear regression

- A common **choice** of loss for regression is a **squared loss function**

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{n}\sum_{i=1}^{n}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

- ***Here***, the optimisation problem can be solved analytically in closed-form. Rewriting the risk matricially, we have

$$R(\boldsymbol{X}, \boldsymbol{\theta}) = \frac{1}{n}\|\boldsymbol{\Phi\theta} - \boldsymbol{y}\|_2^2$$

Feature matrix

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_1^{(1)} & \cdots & \phi_{d'}^{(1)} \\ \vdots & \ddots & \vdots \\ \phi_1^{(n)} & \cdots & \phi_{d'}^{(n)} \end{pmatrix} \in \mathbb{R}^{n \times d'}$$

Target vector

$$\boldsymbol{y} = [y_1, \ldots, y_n]^{\mathrm{T}} \in \mathbb{R}^n$$

The analytical minimisation of the squared loss in linear regression gives the unique solution (when $d' < n$) known as ***normal equations***

$$\widehat{\boldsymbol{\theta}} = \left(\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y}$$

Linear regression: illustration in 1D

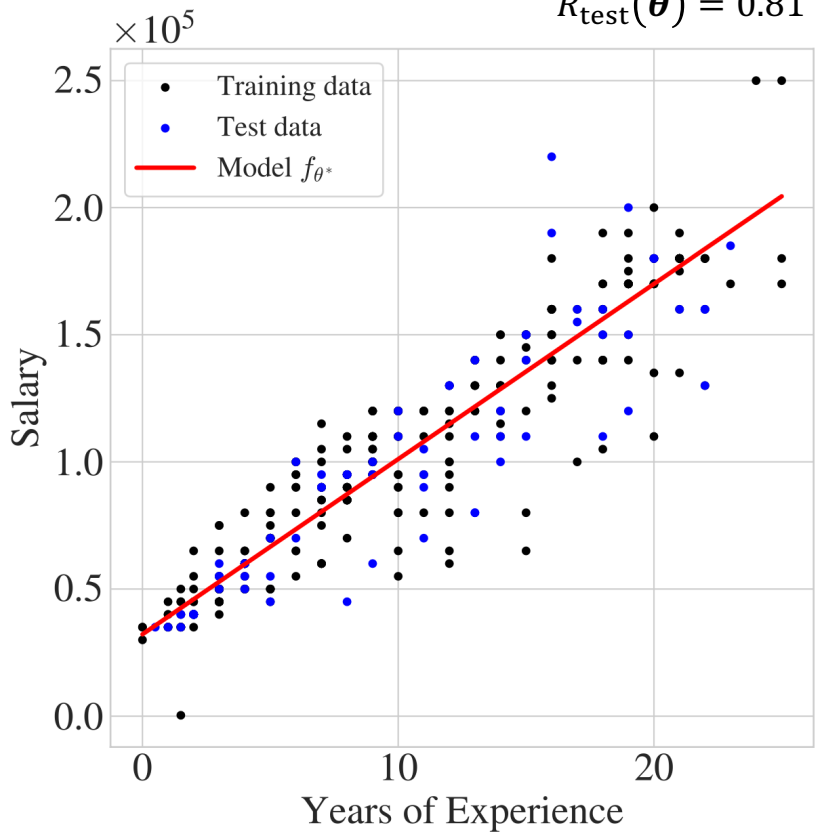
Linear regression

1. I **first** separated the dataset into **training and test sets**, $n = 298$ and $n_{\text{test}} = 75$ chosen randomly.
2. Then, I computed the optimal parameters minimising the empirical risk using the normal equations on the training features

$$\widehat{\boldsymbol{\theta}} = \left(\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\right)^{-1}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y}.$$

3. I computed the risk on the train and test sets and found they are close.

$R_{\text{train}}(\widehat{\boldsymbol{\theta}}) = 0.87$
$R_{\text{test}}(\widehat{\boldsymbol{\theta}}) = 0.81$



\+ Exactly solvable model, low variance

\− Cannot represent local relationships, may be biased

Linear regression

Remark

- The choice of the squared loss can also be motivated from a probabilistic point of view
- Assuming a Gaussian distribution for the error $\epsilon^{(i)} = \hat{y}^{(i)} - y^{(i)} \sim \mathcal{N}(0, \sigma^2)$ and **independent** observations, the ***likelihood*** can be written

$$p(\boldsymbol{X}|\boldsymbol{\theta}) = \prod_i p\left(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{\theta}\right)$$

- Maximising the log-likelihood to obtain the parameters of the model gives

$$\max_\theta \log p(\boldsymbol{X}|\boldsymbol{\theta}) = \max_\theta -\frac{1}{2\sigma_\epsilon^2} \sum_i \left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

→ **The maximum likelihood estimator (MLE) is the same as the empirical risk minimiser under a squared loss function to measure the error of the model**

Linear classification: the model

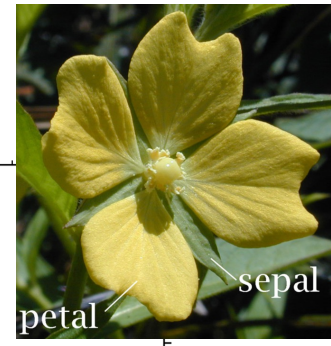
Linear classification

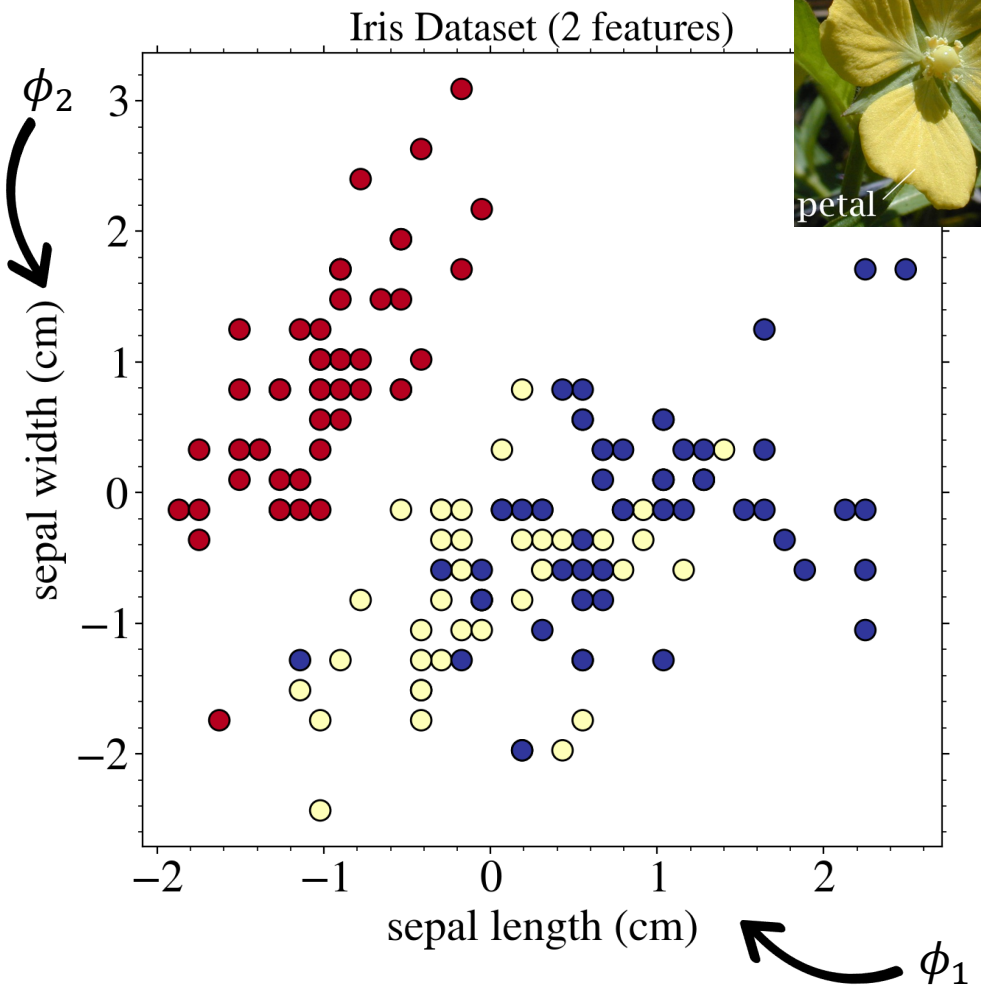
- Consider a K-class classification problem for which features $\boldsymbol{\phi}$ allow linear separability
- Example: Iris dataset with 150 couples $(\phi^{(i)}, y^{(i)}) \Longrightarrow$ **Supervised learning**
- The target variable $y \in \{0,1,2\} \Longrightarrow$ **Classification**
- A natural loss function for classification is the **0-1 loss**

$$\ell(y, \hat{y}) = \begin{cases} 1 & \text{if } \hat{y}^{(i)} \neq y^{(i)}, \\ 0 & \text{otherwise.} \end{cases}$$

- The optimal decision rule (in Bayes sense) is

$$\hat{y} = \operatorname{argmax}_k p(y = k|\boldsymbol{\phi}).$$





We thus need a **model** $p_{\boldsymbol{\theta}}(y = k|\boldsymbol{\phi})$ of the conditional probability distribution to perform classification!

Linear classification: the model

Linear classification

- The simplest models assume a linear log probability

$$\log p_\theta\left(y^{(i)} = k \middle| \boldsymbol{\phi}^{(i)}\right) = \boldsymbol{\theta}_k^{\mathrm{T}} \boldsymbol{\phi}^{(i)} - \log Z$$

where Z is a normalizing constant so that probabilities sum to one.

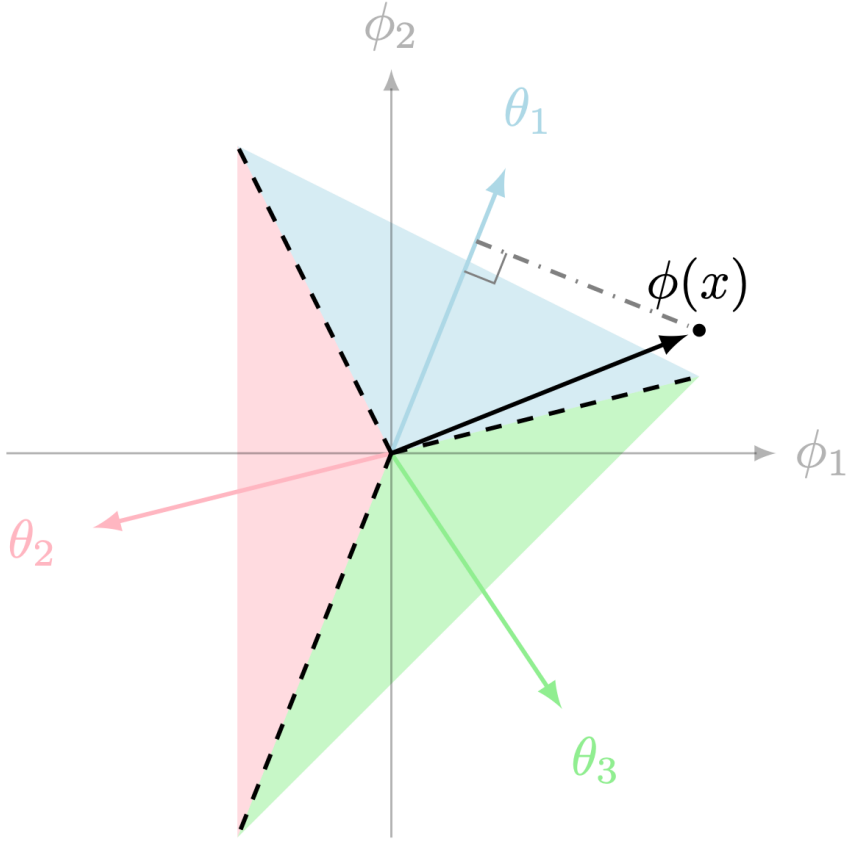
- It means that

$$p_\theta\left(y^{(i)} = k \middle| \boldsymbol{\phi}^{(i)}\right) = \frac{\exp\left(\boldsymbol{\theta}_k^{\mathrm{T}} \boldsymbol{\phi}^{(i)}\right)}{\sum_{j=1}^{K} \exp\left(\boldsymbol{\theta}_j^{\mathrm{T}} \boldsymbol{\phi}^{(i)}\right)}$$

which is called the **softmax function** allowing to turn the linear responses for each class into probabilities.

- And the classification rule is

$$\hat{y} = \mathrm{argmax}_k \, \boldsymbol{\theta}_k^{\mathrm{T}} \boldsymbol{\phi}^{(i)}$$



Geometrically, it corresponds to computing the overlap between the feature $\boldsymbol{\phi}^{(i)}$ and a vector representative for each class, $\boldsymbol{\theta}_k$, and associating **the class maximising the dot product**, leading to **linear decision** boundaries shown as hyperplanes.

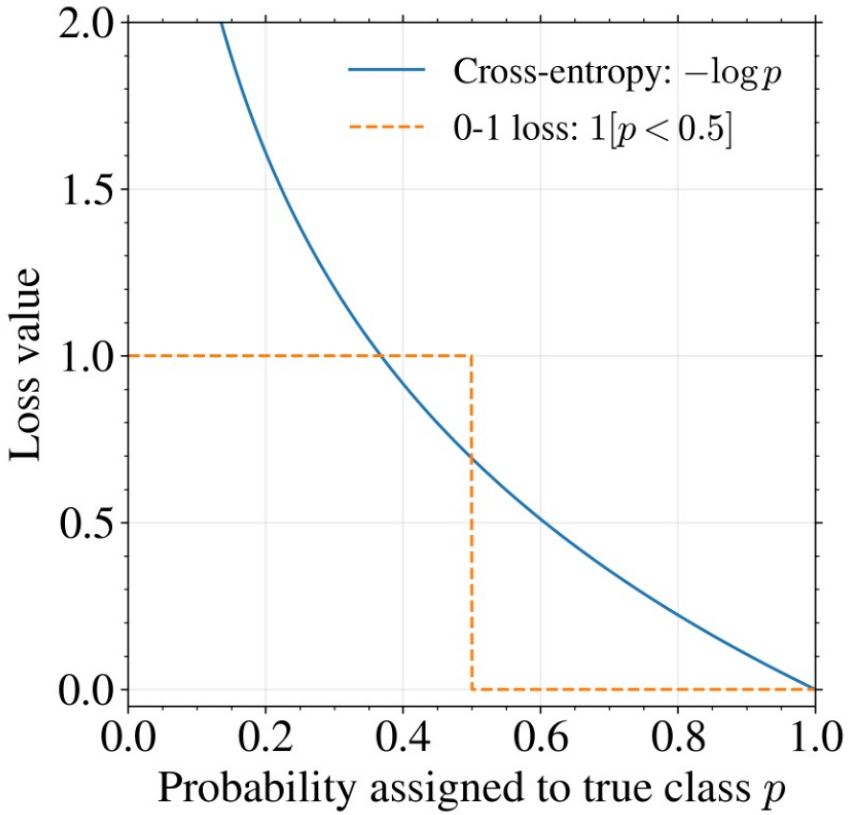
Linear classification

- Now the model is specified, we need minimise the risk to obtain the parameters $\boldsymbol{\theta}_k$ using some training data

- For optimisation, the 0-1 loss is not suitable since it is not differentiable, but we can relax it using the probabilities

$$\ell(y, \hat{y}) = -\sum_{k=1}^{K} 1_{y=k} \log p_\theta(y = k|\boldsymbol{\phi}^{(i)})$$

which is **now differentiable and convex.**



Linear classification: fitting parameters

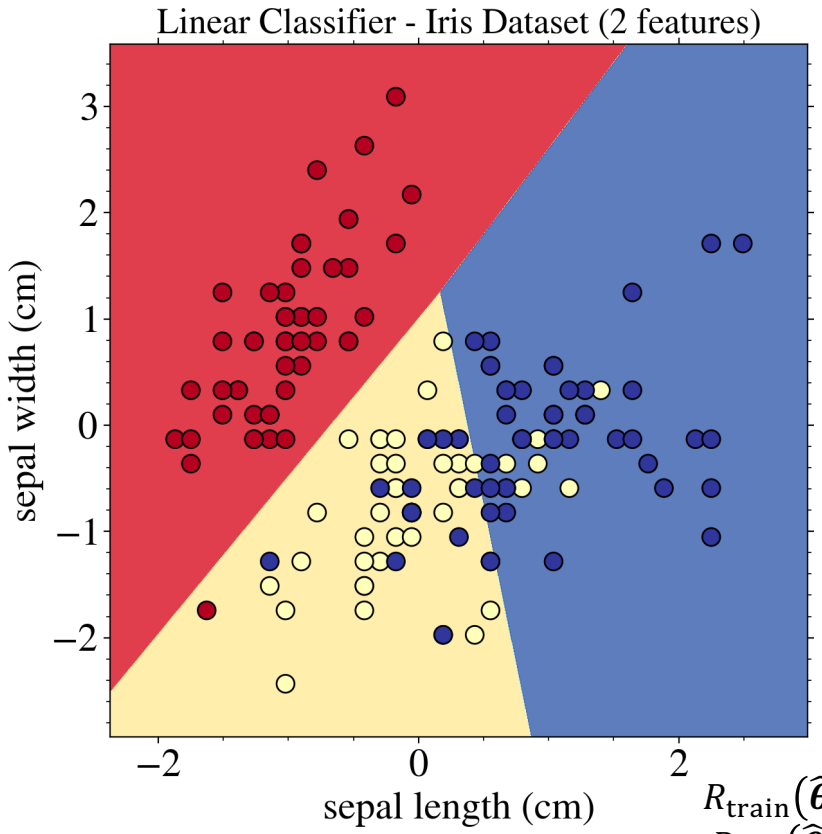
Linear classification

- Now the model is specified, we need minimise the risk to obtain the parameters $\boldsymbol{\theta}_k$ using some training data

- For optimisation, the 0-1 loss is not suitable since it is not differentiable, but we can relax it using the probabilities

$$\ell(y, \hat{y}) = -\sum_{k=1}^{K} 1_{y=k} \log p_\theta(y = k|\boldsymbol{\phi}^{(i)})$$

which is **now differentiable and convex.**



$R_{\text{train}}(\widehat{\boldsymbol{\theta}}) = 0.38$
$R_{\text{test}}(\widehat{\boldsymbol{\theta}}) = 0.48$

This risk is referred to as ***cross-entropy*** and it is the most widely used cost function for classification problems. The parameters of the model are then obtained by minimising the risk, i.e.

$$\widehat{\boldsymbol{\theta}} = \text{argmin}_\theta \, R(\boldsymbol{X}, \boldsymbol{\theta}).$$